

Felipe Fazio da Costa, RA: 23.00055-4
João Gabriel Fioruci Roberto, RA: 23.00617-0
Gabriel Rodrigues Marques, RA: 23.00578-5
Fábio Sadao Sato, RA: 22.00984-0

Projeto Semestral: Carro de Bombeiro Teleoprado

Brasil

2026

Felipe Fazio da Costa, RA: 23.00055-4
João Gabriel Fioruci Roberto, RA: 23.00617-0
Gabriel Rodrigues Marques, RA: 23.00578-5
Fábio Sadao Sato, RA: 22.00984-0

Projeto Semestral: Carro de Bombeiro Teleoprado

Relatório técnico apresentado como requisito
parcial para aprovação na disciplina EEN251
— Projeto Integrador de Sistemas Embarcados
do curso de Engenharia.

Centro Universitário Instituto Mauá de Tecnologia
Engenharia de Computação / Eletrônica
Disciplina: EEN251 — Projeto de Sistemas Embarcados

Brasil
2026

Resumo

Este relatório descreve o desenvolvimento de um protótipo de carro de bombeiro teleoprado utilizando microcontroladores Raspberry Pi Pico e comunicação via rádio frequência (NRF24L01). O sistema integra tração 4WD, sistema de combate a incêndio por bomba d'água e monitoramento de nível de reservatório. Foram aplicados conceitos de sistemas embarcados, comunicação sem fio e controle de periféricos via PWM. Os resultados demonstram a eficácia da plataforma RP2040 em aplicações de controle em tempo real e teleoperação robusta.

Palavras-chave: Raspberry Pi Pico. NRF24L01. MicroPython. Carro de Bombeiro. Sistemas Embarcados.

Abstract

This report describes the development of a teleoperated fire truck prototype using Raspberry Pi Pico microcontrollers and radio frequency communication (NRF24L01). The system integrates 4WD traction, a water pump fire suppression system, and reservoir level monitoring. Concepts of embedded systems, wireless communication, and peripheral control via PWM were applied. The results demonstrate the effectiveness of the RP2040 platform in real-time control applications and robust teleoperation.

Keywords: Raspberry Pi Pico. NRF24L01. MicroPython. Fire Truck. Embedded Systems.

Sumário

1	INTRODUÇÃO	5
1.1	Objetivos	5
2	DESENVOLVIMENTO DO PROJETO	6
2.1	Arquitetura e Diagramas de Blocos	6
2.1.1	Subsistema Transmissor (Controle)	6
2.1.2	Subsistema Receptor (Veículo)	6
2.2	Arquitetura de Hardware	7
2.2.1	Transmissor (Controle)	7
2.2.2	Receptor (Veículo)	7
2.3	Comunicação RF (NRF24L01)	7
2.4	Software e Firmware	8
2.4.1	Lógica de Movimentação (Tank Drive)	8
2.4.2	Segurança e Failsafe	8
2.5	Modelagem 3D e Estrutura Física	8
2.6	Esquemas Elétricos e PCB	8
3	CONCLUSÃO	11
	REFERÊNCIAS BIBLIOGRÁFICAS	12
	APÊNDICES	13
	APÊNDICE A – CÓDIGO FONTE DO RECEPTOR	14
	APÊNDICE B – CÓDIGO FONTE DO TRANSMISSOR	20
	ANEXOS	29
	ANEXO A – ESQUEMA ELÉTRICO DO CARRO	30
	ANEXO B – LAYOUT DA PCB DO CARRO	32
	ANEXO C – ESQUEMA ELÉTRICO DO CONTROLE	36
	ANEXO D – LAYOUT DA PCB DO CONTROLE	38

1 Introdução

O projeto semestral da disciplina EEN251 — Projeto Integrador de Sistemas Embarcados propõe o desenvolvimento de um sistema embarcado completo, integrando hardware e software. O tema escolhido foi um carro de bombeiro teleoperado, que combina mobilidade, atuação remota e sensoriamento.

O uso de veículos teleoperados é comum em situações de risco, como o combate a incêndios, onde a segurança humana é prioridade. O protótipo desenvolvido busca simular essas funcionalidades em escala reduzida.

1.1 Objetivos

O objetivo geral deste trabalho é projetar e construir um veículo robótico controlado remotamente via radiofrequência, capaz de realizar manobras de locomoção e acionamento de uma bomba d'água, com monitoramento de status em tempo real.

Como objetivos específicos, destacam-se:

- Implementação de comunicação bidirecional robusta entre controle e veículo;
- Controle de tração diferencial (tank drive) para quatro motores;
- Sistema de alerta de nível de água com indicadores visuais e sonoros;
- Desenvolvimento de firmware em MicroPython para a plataforma Raspberry Pi Pico.

2 Desenvolvimento do Projeto

O desenvolvimento foi dividido em três frentes principais: hardware, software e integração de sistemas.

2.1 Arquitetura e Diagramas de Blocos

A Figura 1 apresenta a visão sistêmica do projeto, destacando a separação entre as unidades de controle e de atuação no veículo.

Figura 1 – Diagrama de Blocos do Sistema Completo

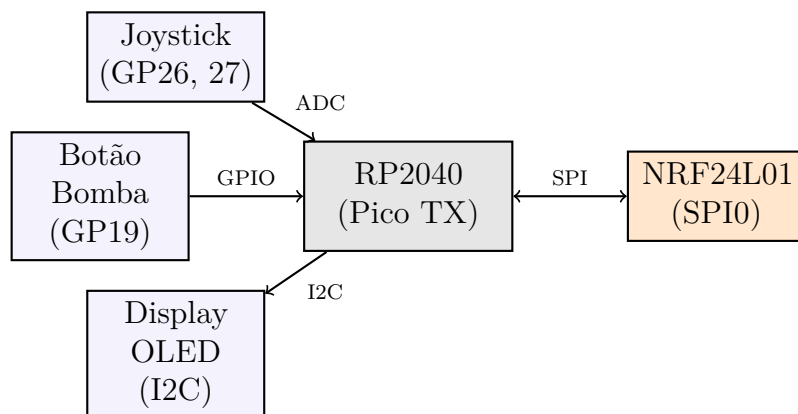


Fonte: Os autores (2026)

2.1.1 Subsistema Transmissor (Controle)

O controle remoto (Figura 2) processa as entradas do joystick e botões através do Raspberry Pi Pico, transmitindo-as via NRF24L01.

Figura 2 – Fluxo de Dados do Transmissor

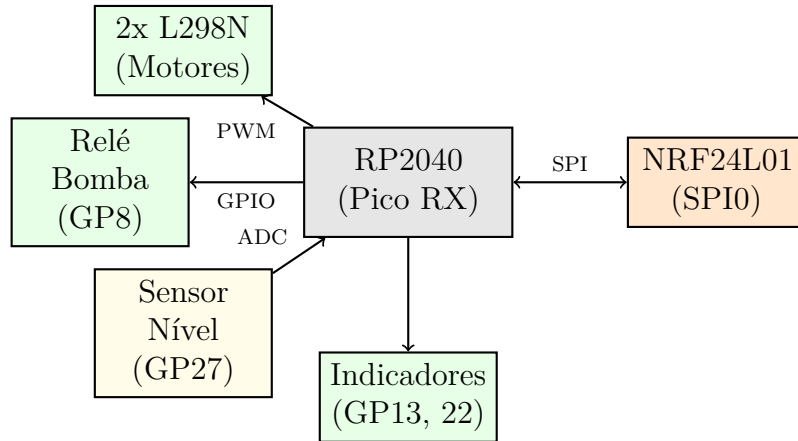


Fonte: Os autores (2026)

2.1.2 Subsistema Receptor (Veículo)

O receptor (Figura 3) traduz os sinais RF em comandos de potência para os motores e gerencia os sensores de campo.

Figura 3 – Fluxo de Atuação do Receptor



Fonte: Os autores (2026)

2.2 Arquitetura de Hardware

O sistema é composto por dois nós principais: o Transmissor (Controle) e o Receptor (Veículo). Ambos utilizam o microcontrolador RP2040 ([RASPERRY PI LTD, 2026b](#)), presente na placa Raspberry Pi Pico ([RASPERRY PI LTD, 2026a](#)).

2.2.1 Transmissor (Controle)

O controle remoto utiliza um joystick analógico para comandos de direção e botões para acionamento da bomba. Um display OLED SSD1306 foi integrado para fornecer feedback visual ao operador.

2.2.2 Receptor (Veículo)

O veículo possui um chassi 4WD acionado por motores DC. O controle de potência é realizado por drivers L298N ([STMICROELECTRONICS, 2026](#)). O sistema possui uma mini bomba submersa acionada por relé.

2.3 Comunicação RF (NRF24L01)

A comunicação utiliza o módulo NRF24L01 ([NORDIC SEMICONDUCTOR, 2026](#)) operando em 2.4 GHz. Foi adotado o protocolo Enhanced ShockBurst. Todo o desenvolvimento do rádio foi baseado na biblioteca ([MICROPYTHON-NRF24L01..., 2026](#)).

2.4 Software e Firmware

Todo o código foi desenvolvido em MicroPython ([MICROPYTHON PROJECT, 2026](#)). O firmware gerencia a recepção de pacotes e o controle de periféricos.

2.4.1 Lógica de Movimentação (Tank Drive)

```
1 def drive(x, y):  
2     motor_esq = max(-100, min(100, y + x))  
3     motor_dir = max(-100, min(100, y - x))  
4     set_motor(motor_esq, motor_dir)
```

Listing 2.1 – Algoritmo de Tank Drive em MicroPython

2.4.2 Segurança e Failsafe

```
1 if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > 500:  
2     parar()           # Desliga motores por seguranca
```

Listing 2.2 – Logica de Watchdog

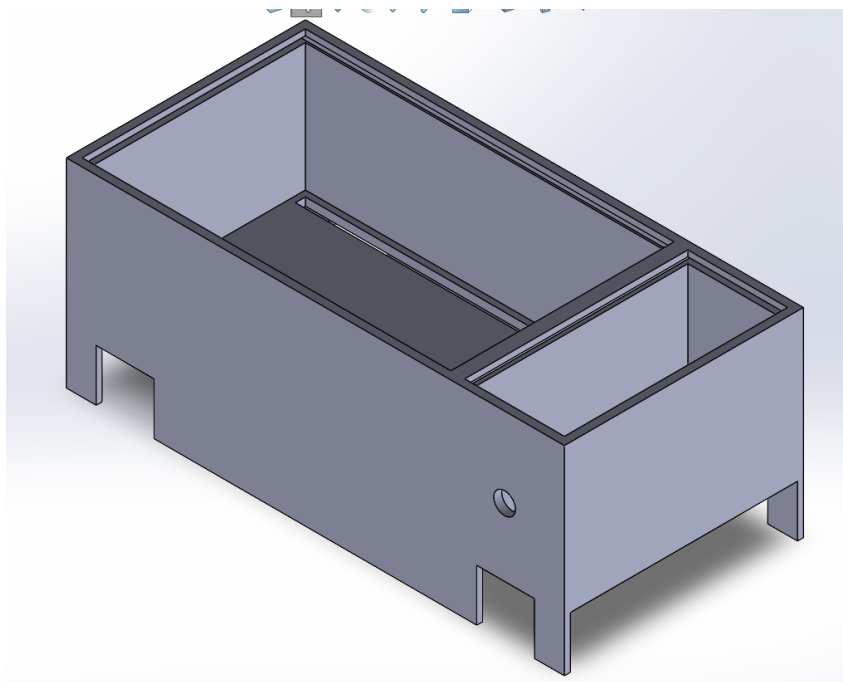
2.5 Modelagem 3D e Estrutura Física

A estrutura física foi projetada em CAD. A Figura 4 apresenta a renderização do veículo.

2.6 Esquemas Elétricos e PCB

A Figura 7 ilustra a visualização 3D da PCB do carro, enquanto a Figura 8 apresenta a visualização 3D da PCB do controle.

Figura 4 – Modelagem 3D do Carro de Bombeiro



Fonte: Os autores (2026)

Figura 5 – Base do Controle

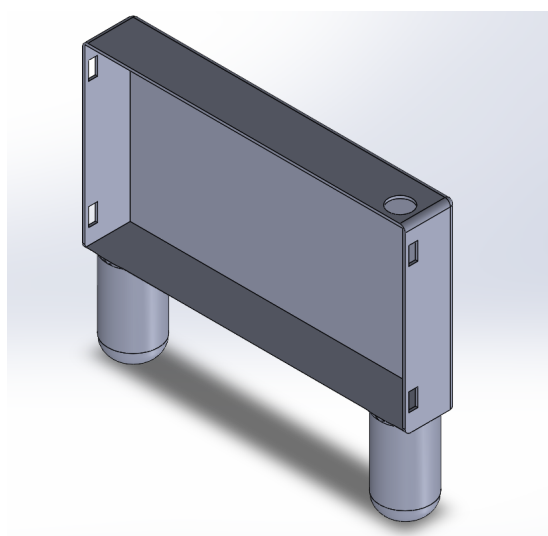
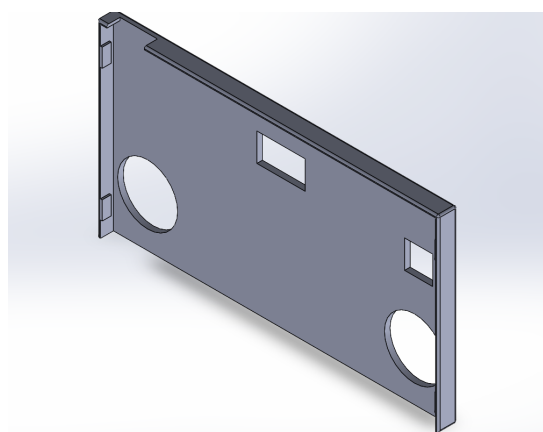
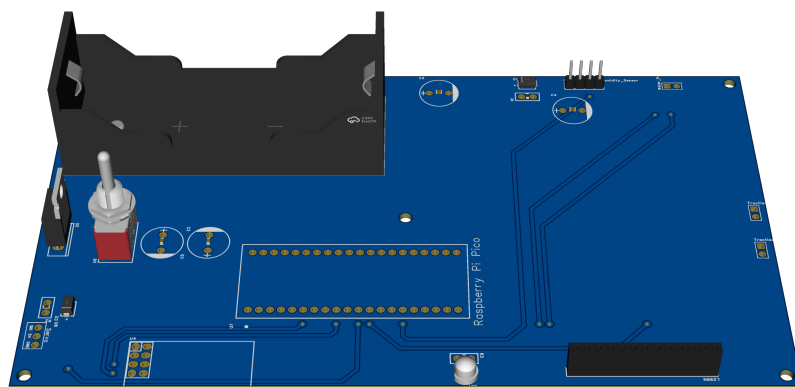


Figura 6 – Tampa do Controle



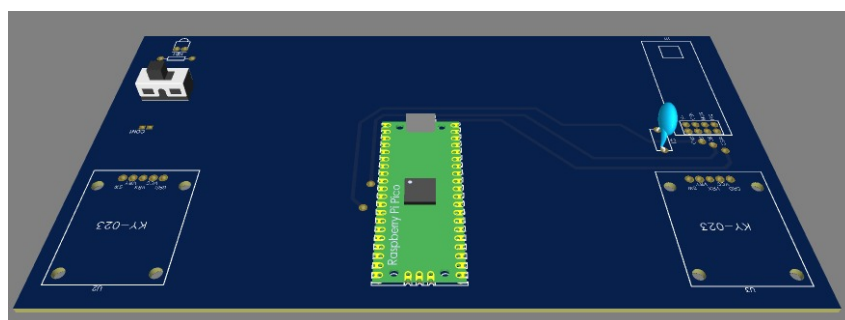
Fonte: Os autores (2026)

Figura 7 – Visualização 3D da PCB do Carro



Fonte: Os autores (2026)

Figura 8 – Visualização 3D da PCB do Controle



Fonte: Os autores (2026)

3 Conclusão

O projeto atingiu os objetivos. A integração NRF24L01/RP2040 permitiu uma teleoperação robusta e segura.

Referências Bibliográficas

MICROPYTHON-NRF24L01 Library. 2026. Acesso em: mar. 2026. Disponível em: <https://github.com/micropython/micropython-lib/tree/master/micropython/drivers/radio/nrf24l01>>. Citado na página 7.

MICROPYTHON PROJECT. **MicroPython Documentation for RP2**. 2026. Acesso em: mar. 2026. Disponível em: <https://docs.micropython.org/en/latest/rp2/quickref.html>>. Citado na página 8.

NORDIC SEMICONDUCTOR. **nRF24L01+ Product Specification v1.0**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://www.nordicsemi.com/products/nrf24l01>>. Citado na página 7.

RASPBERRY PI LTD. **Raspberry Pi Pico Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>>. Citado na página 7.

RASPBERRY PI LTD. **RP2040 Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>>. Citado na página 7.

STMICROELECTRONICS. **L298N Dual Full-Bridge Driver Datasheet**. [S.l.], 2026. Acesso em: mar. 2026. Disponível em: <https://www.st.com/resource/en/datasheet/l298.pdf>>. Citado na página 7.

Apêndices

APÊNDICE A – Código Fonte do Receptor

```

1 from machine import Pin, SPI, PWM, ADC
2 import time
3
4 # =====
5 # SPI NRF24L01
6 # =====
7 spi = SPI(0,
8           baudrate=1000000,
9           polarity=0,
10          phase=0,
11          sck=Pin(6),
12          mosi=Pin(7),
13          miso=Pin(4))
14 csn = Pin(5, Pin.OUT)
15 ce = Pin(12, Pin.OUT)
16
17 # =====
18 # L298N - PONTE H
19 # =====
20 in1 = Pin(0, Pin.OUT)
21 in2 = Pin(1, Pin.OUT)
22 ena = PWM(Pin(9))
23 ena.freq(1000)
24
25 in3 = Pin(2, Pin.OUT)
26 in4 = Pin(3, Pin.OUT)
27 enb = PWM(Pin(10))
28 enb.freq(1000)
29
30 in1.value(0)
31 in2.value(0)
32 in3.value(0)
33 in4.value(0)
34 ena.duty_u16(0)
35 enb.duty_u16(0)
36
37 # =====
38 # BOMBA D'ÁGUA - GP8
39 # =====
40 bomba = Pin(8, Pin.OUT)
41 bomba.value(0)
42
43 # LED indicador

```

```
44 led = Pin(13, Pin.OUT)
45
46 # =====
47 # SENSOR DE UMIDADE
48 # =====
49 umidade_analog = ADC(Pin(27))
50 umidade_digital = Pin(22, Pin.IN)
51
52 FORA = 60000
53 SUBMERSO = 10000
54
55 def ler_umidade():
56     raw = sum(umidade_analog.read_u16() for _ in range(10)) // 10
57     nivel = (FORA - raw) * 100 // (FORA - SUBMERSO)
58     nivel = max(0, min(100, nivel))
59     digital = 1 if umidade_digital.value() == 0 else 0
60     return nivel, digital
61
62 # =====
63 # ENDEREÇOS
64 # =====
65 ADDR_CONTROLE = [0xE7, 0xE7, 0xE7, 0xE7, 0xE7]
66 ADDR_CARRO = [0xC2, 0xC2, 0xC2, 0xC2, 0xC2]
67
68 # =====
69 # DRIVER NRF24L01
70 # =====
71 class NRF24:
72     def __init__(self, spi, csn, ce):
73         self.spi = spi
74         self.csn = csn
75         self.ce = ce
76         self.csn.value(1)
77         self.ce.value(0)
78         self.init_rx()
79
80     def write_reg(self, reg, value):
81         self.csn.value(0)
82         self.spi.write(bytearray([0x20 | reg, value]))
83         self.csn.value(1)
84
85     def read_reg(self, reg):
86         self.csn.value(0)
87         self.spi.write(bytearray([reg & 0x1F]))
88         result = self.spi.read(1)
89         self.csn.value(1)
90         return result[0]
```



```
91
92     def set_addr(self, pipe_reg, addr_bytes):
93         self.csn.value(0)
94         self.spi.write(bytearray([0x20 | pipe_reg]) + bytearray(
addr_bytes))
95         self.csn.value(1)
96
97     def flush_rx(self):
98         self.csn.value(0)
99         self.spi.write(bytearray([0xE2]))
100        self.csn.value(1)
101
102     def flush_tx(self):
103         self.csn.value(0)
104         self.spi.write(bytearray([0xE1]))
105        self.csn.value(1)
106
107     def init_rx(self):
108         time.sleep_ms(100)
109         self.write_reg(0x00, 0x0B)    # Power up + RX
110         self.write_reg(0x01, 0x00)    # Sem auto-ack
111         self.write_reg(0x02, 0x01)    # Pipe 0 habilitado
112         self.write_reg(0x05, 0x02)    # Canal 2
113         self.write_reg(0x06, 0x26)    # Data rate 1Mbps
114
115         self.set_addr(0x0A, ADDR_CARRO) # RX pipe 0 addr
116         self.set_addr(0x10, ADDR_CONTROLE) # TX addr (resposta)
117         self.write_reg(0x11, 4)         # Payload pipe 0 = 4 bytes
118
119         self.flush_rx()
120         self.flush_tx()
121         self.write_reg(0x07, 0x70)
122         self.ce.value(1)
123
124     def modo_tx(self):
125         self.ce.value(0)
126         self.write_reg(0x00, 0x0A)
127         self.set_addr(0x10, ADDR_CONTROLE) # TX aponta para o controle
128         self.flush_tx()
129         time.sleep_us(150)
130
131     def modo_rx(self):
132         self.ce.value(0)
133         self.write_reg(0x00, 0x0B)
134         self.set_addr(0x0A, ADDR_CARRO)
135         self.flush_rx()
136         self.write_reg(0x07, 0x70)
```

```
137         self.ce.value(1)
138         time.sleep_us(150)
139
140     def available(self):
141         status = self.read_reg(0x07)
142         if status & 0x10:
143             self.write_reg(0x07, 0x70)
144             self.flush_rx()
145         return (status & 0x40) != 0
146
147     def receive(self):
148         self.csn.value(0)
149         self.spi.write(bytearray([0x61]))
150         data = self.spi.read(4)
151         self.csn.value(1)
152         self.write_reg(0x07, 0x40)
153         return data
154
155     def send(self, data):
156         self.mod0_tx()
157         self.csn.value(0)
158         self.spi.write(bytearray([0xA0]) + data)
159         self.csn.value(1)
160         self.ce.value(1)
161         time.sleep_us(20)
162         self.ce.value(0)
163         time.sleep_ms(5)
164         self.write_reg(0x07, 0x70)
165         self.mod0_rx()
166
167     def reset(self):
168         self.ce.value(0)
169         self.flush_rx()
170         self.flush_tx()
171         self.write_reg(0x07, 0x70)
172         self.mod0_rx()
173         print("Radio reiniciado!")
174
175     # =====
176     # FUNÇÕES DOS MOTORES
177     # =====
178     def set_motor(in_a, in_b, en_pwm, speed):
179         speed = max(-100, min(100, speed))
180         if speed >= 10:
181             in_a.value(1)
182             in_b.value(0)
183             en_pwm.duty_u16(int(speed / 100 * 65535))
```

```

184     elif speed <= -10:
185         in_a.value(0)
186         in_b.value(1)
187         en_pwm.duty_u16(int(abs(speed) / 100 * 65535))
188     else:
189         in_a.value(0)
190         in_b.value(0)
191         en_pwm.duty_u16(0)
192
193 def drive(x, y):
194     motor_esq = max(-100, min(100, y + x))
195     motor_dir = max(-100, min(100, y - x))
196     set_motor(in1, in2, ena, motor_esq)
197     set_motor(in3, in4, enb, motor_dir)
198
199 def parar():
200     set_motor(in1, in2, ena, 0)
201     set_motor(in3, in4, enb, 0)
202
203 # =====
204 # INICIALIZA RÁDIO
205 # =====
206 radio = NRF24(spi, csn, ce)
207 print("Receptor pronto, aguardando sinal...")
208 radio.modos_rx() # Garantir RX mode ao iniciar
209
210 # Debug: mostrar config inicial
211 print(">>> CONFIG INICIAL (RX):")
212 print(f"CONFIG: {radio.read_reg(0x00):#04x} | EN_RX: {radio.read_reg(0x02):#04x} | CH: {radio.read_reg(0x05):#04x}")
213
214 # =====
215 # LOOP PRINCIPAL
216 # =====
217 TIMEOUT = 500
218 TIMEOUT_RESET = 2000
219
220 ultimo_sinal = time.time()
221 primeiro_dump = True
222
223 while True:
224     if radio.available():
225         data = radio.receive()
226         t0 = time.time()
227
228         y = data[0] if data[0] < 128 else data[0] - 256
229         x = data[1] if data[1] < 128 else data[1] - 256

```

```
230     btn = data[2]
231
232     drive(x, y)
233     bomba.value(1 if btn == 0 else 0)
234     led.value(1)
235     ultimo_sinal = time.ticks_ms()
236
237     umid_pct, umid_dig = ler_umidade()
238     t1 = time.ticks_ms()
239
240     payload_umid = bytearray([
241         umid_pct & 0xFF,
242         umid_dig & 0xFF,
243         0xFF,
244         0
245     ])
246
247     radio.send(payload_umid)
248     time.sleep_ms(50) # Aumentado de 15ms → 50ms para garantir que
resposta chega ao controle
249     radio.modem_rx() # Voltar para RX mode
250     t2 = time.ticks_ms()
251
252     print(f"X: {x} | Y: {y} | BOMBA: {1 if btn == 0 else 0} | "
253           f"Nivel: {umid_pct}% | {'AGUA' if umid_dig == 1 else 'SECO'
254           },}")
255
256     print(f"RESPOSTA_TX | Umidade: {umid_pct}% | Tempo ate envio: {
time.ticks_diff(t1, t0)}ms | Total: {time.ticks_diff(t2, t0)}ms")
257
258     if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > TIMEOUT:
259         parar()
260         bomba.value(0)
261         led.value(0)
262
263     if time.ticks_diff(time.ticks_ms(), ultimo_sinal) > TIMEOUT_RESET:
264         radio.reset()
265         ultimo_sinal = time.ticks_ms()
266
267     time.sleep_ms(10)
```

APÊNDICE B – Código Fonte do Transmissor

```

1 from machine import Pin, ADC, SPI, I2C
2 import time
3
4 # =====
5 # CONFIG GPIO
6 # =====
7 joy1_x = ADC(27)
8 joy1_y = ADC(26)
9 button = Pin(19, Pin.IN, Pin.PULL_UP)
10 led     = Pin(25, Pin.OUT)
11
12 # =====
13 # I2C + DISPLAY OLED SSD1306
14 # =====
15 i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
16
17 class SSD1306:
18     def __init__(self, i2c, addr=0x3C, width=128, height=64):
19         self.i2c      = i2c
20         self.addr      = addr
21         self.width     = width
22         self.height    = height
23         self.pages     = height // 8
24         self.buffer    = bytearray(self.pages * width)
25         self._init_display()
26
27     def _cmd(self, cmd):
28         self.i2c.writeto(self.addr, bytearray([0x00, cmd]))
29
30     def _init_display(self):
31         for cmd in [
32             0xAE, 0x20, 0x00, 0xB0, 0xC8,
33             0x00, 0x10, 0x40, 0x81, 0xFF,
34             0xA1, 0xA6, 0xA8, 0x3F, 0xA4,
35             0xD3, 0x00, 0xD5, 0xF0, 0xD9,
36             0x22, 0xDA, 0x12, 0xDB, 0x20,
37             0x8D, 0x14, 0xAF
38         ]:
39             self._cmd(cmd)
40

```

```

41     def show(self):
42         for page in range(self.pages):
43             self._cmd(0xB0 + page)
44             self._cmd(0x00)
45             self._cmd(0x10)
46             self.i2c.writeto(self.addr,
47                             bytearray([0x40]) + self.buffer[(page * self.width:(page
+ 1) * self.width]))
48
49     def fill(self, color):
50         val = 0xFF if color else 0x00
51         self.buffer = bytearray([val] * len(self.buffer))
52
53     def pixel(self, x, y, color):
54         if 0 <= x < self.width and 0 <= y < self.height:
55             page = y // 8
56             bit = y % 8
57             idx = page * self.width + x
58             if color:
59                 self.buffer[idx] |= (1 << bit)
60             else:
61                 self.buffer[idx] &= ~(1 << bit)
62
63     FONT = {
64         ' ': [0,0,0,0,0],
65         'A': [0x7E,0x09,0x09,0x09,0x7E],
66         'B': [0x7F,0x49,0x49,0x49,0x36],
67         'C': [0x3E,0x41,0x41,0x41,0x22],
68         'D': [0x7F,0x41,0x41,0x22,0x1C],
69         'E': [0x7F,0x49,0x49,0x49,0x41],
70         'F': [0x7F,0x09,0x09,0x09,0x01],
71         'G': [0x3E,0x41,0x49,0x49,0x7A],
72         'H': [0x7F,0x08,0x08,0x08,0x7F],
73         'I': [0x41,0x7F,0x41,0x00,0x00],
74         'J': [0x20,0x40,0x41,0x3F,0x01],
75         'K': [0x7F,0x08,0x14,0x22,0x41],
76         'L': [0x7F,0x40,0x40,0x40,0x40],
77         'M': [0x7F,0x02,0x0C,0x02,0x7F],
78         'N': [0x7F,0x04,0x08,0x10,0x7F],
79         'O': [0x3E,0x41,0x41,0x41,0x3E],
80         'P': [0x7F,0x09,0x09,0x09,0x06],
81         'Q': [0x3E,0x41,0x51,0x21,0x5E],
82         'R': [0x7F,0x09,0x19,0x29,0x46],
83         'S': [0x46,0x49,0x49,0x49,0x31],
84         'T': [0x01,0x01,0x7F,0x01,0x01],
85         'U': [0x3F,0x40,0x40,0x40,0x3F],
86         'V': [0x1F,0x20,0x40,0x20,0x1F],

```

```

87         'W': [0x3F,0x40,0x38,0x40,0x3F],
88         'X': [0x63,0x14,0x08,0x14,0x63],
89         'Y': [0x07,0x08,0x70,0x08,0x07],
90         'Z': [0x61,0x51,0x49,0x45,0x43],
91         '0': [0x3E,0x51,0x49,0x45,0x3E],
92         '1': [0x00,0x42,0x7F,0x40,0x00],
93         '2': [0x42,0x61,0x51,0x49,0x46],
94         '3': [0x21,0x41,0x45,0x4B,0x31],
95         '4': [0x18,0x14,0x12,0x7F,0x10],
96         '5': [0x27,0x45,0x45,0x45,0x39],
97         '6': [0x3C,0x4A,0x49,0x49,0x30],
98         '7': [0x01,0x71,0x09,0x05,0x03],
99         '8': [0x36,0x49,0x49,0x49,0x36],
100        '9': [0x06,0x49,0x49,0x29,0x1E],
101        ':': [0x00,0x36,0x36,0x00,0x00],
102        '-': [0x08,0x08,0x08,0x08,0x08],
103        '%': [0x23,0x13,0x08,0x64,0x62],
104    }
105
106    def text(self, string, x, y, color=1):
107        for ch in string.upper():
108            glyph = self.FONT.get(ch, self.FONT[' '])
109            for col_idx, col in enumerate(glyph):
110                for row in range(8):
111                    self.pixel(x + col_idx, y + row, (col >> row) & 1 if
color else 0)
112                x += 6
113                if x > self.width:
114                    break
115
116    # Inicializa display
117    display = SSD1306(i2c)
118    display.fill(0)
119    display.text("CARRO", 30, 20)
120    display.text("BOMBEIRO", 16, 36)
121    display.show()
122    time.sleep(2)
123
124    # =====
125    # SPI NRF24L01
126    # =====
127    spi = SPI(0,
128                baudrate=1000000,
129                polarity=0,
130                phase=0,
131                sck=Pin(6),
132                mosi=Pin(7),

```

```
133         miso=Pin(4))
134 csn = Pin(5, Pin.OUT)
135 ce  = Pin(9, Pin.OUT)
136
137 # =====
138 # ENDEREÇOS
139 # =====
140 ADDR_CONTROLE = [0xE7,0xE7,0xE7,0xE7,0xE7]
141 ADDR_CARRO    = [0xC2,0xC2,0xC2,0xC2,0xC2]
142
143 # =====
144 # DRIVER NRF24L01
145 # =====
146 class NRF24:
147     def __init__(self, spi, csn, ce):
148         self.spi = spi
149         self.csn = csn
150         self.ce  = ce
151         self.csn.value(1)
152         self.ce.value(0)
153         self.init_tx()
154
155     def write_reg(self, reg, value):
156         self.csn.value(0)
157         self.spi.write(bytearray([0x20 | reg, value]))
158         self.csn.value(1)
159
160     def read_reg(self, reg):
161         self.csn.value(0)
162         self.spi.write(bytearray([reg & 0x1F]))
163         result = self.spi.read(1)
164         self.csn.value(1)
165         return result[0]
166
167     def set_addr(self, pipe_reg, addr_bytes):
168         self.csn.value(0)
169         self.spi.write(bytearray([0x20 | pipe_reg]) + bytearray(
addr_bytes))
170         self.csn.value(1)
171
172     def flush_rx(self):
173         self.csn.value(0)
174         self.spi.write(bytearray([0xE2]))
175         self.csn.value(1)
176
177     def flush_tx(self):
178         self.csn.value(0)
```



```
179         self.spi.write(bytearray([0xE1]))
180         self.csn.value(1)
181
182     def init_tx(self):
183         time.sleep_ms(100)
184         self.write_reg(0x00, 0x0A)
185         self.write_reg(0x01, 0x00)
186         self.write_reg(0x02, 0x01)
187         self.write_reg(0x05, 0x02)
188         self.write_reg(0x06, 0x26)
189         self.set_addr(0x10, ADDR_CARRO)
190         self.set_addr(0x0A, ADDR_CONTROLE)
191         self.write_reg(0x11, 4)
192         self.flush_rx()
193         self.flush_tx()
194         self.write_reg(0x07, 0x70)
195
196     def modo_rx(self):
197         self.ce.value(0)
198         self.write_reg(0x00, 0x0B)
199         self.write_reg(0x02, 0x01)
200         self.set_addr(0x0A, ADDR_CONTROLE)
201         self.flush_rx()
202         self.write_reg(0x07, 0x70)
203         self.ce.value(1)
204         time.sleep_us(150)
205
206     def modo_tx(self):
207         self.ce.value(0)
208         self.write_reg(0x00, 0x0A)
209         self.flush_tx()
210         time.sleep_us(150)
211
212     def send(self, data):
213         self.modo_tx()
214         self.csn.value(0)
215         self.spi.write(bytearray([0xA0]) + data)
216         self.csn.value(1)
217         self.ce.value(1)
218         time.sleep_us(15)
219         self.ce.value(0)
220         time.sleep_ms(3)
221         self.write_reg(0x07, 0x70)
222         self.modo_rx()
223
224     def available(self):
225         status = self.read_reg(0x07)
```

```

226         if status & 0x10:
227             self.write_reg(0x07, 0x70)
228             self.flush_rx()
229         return (status & 0x40) != 0
230
231     def receive(self):
232         self.csn.value(0)
233         self.spi.write(bytearray([0x61]))
234         data = self.spi.read(4)
235         self.csn.value(1)
236         self.write_reg(0x07, 0x40)
237         return data
238
239     def dump_regs(self):
240         print("=== DUMP REGISTRADORES ===")
241         print(f"CONFIG      (0x00): {self.read_reg(0x00):#04x}")
242         print(f"EN_AA       (0x01): {self.read_reg(0x01):#04x}")
243         print(f"EN_RXADDR  (0x02): {self.read_reg(0x02):#04x}")
244         print(f"RF_CH      (0x05): {self.read_reg(0x05):#04x}")
245         print(f"RF_SETUP   (0x06): {self.read_reg(0x06):#04x}")
246         print(f"STATUS     (0x07): {self.read_reg(0x07):#04x}")
247         print(f"RX_PW_P0   (0x11): {self.read_reg(0x11):#04x}")
248         self.csn.value(0)
249         self.spi.write(bytearray([0x10]))
250         tx_addr = self.spi.read(5)
251         self.csn.value(1)
252         print(f"TX_ADDR    (0x10): {[hex(b) for b in tx_addr]}")
253         self.csn.value(0)
254         self.spi.write(bytearray([0x0A]))
255         rx0_addr = self.spi.read(5)
256         self.csn.value(1)
257         print(f"RX_ADDR_P0(0x0A): {[hex(b) for b in rx0_addr]}")
258         print("=====")
259
260     # =====
261     # INICIALIZA RÁDIO
262     # =====
263     radio = NRF24(spi, csn, ce)
264     print(">>> DUMP INICIAL (modo TX):")
265     radio.dump_regs()
266
267     # =====
268     # FUNÇÕES
269     # =====
270     def read_joystick(adc):
271         return adc.read_u16()
272

```

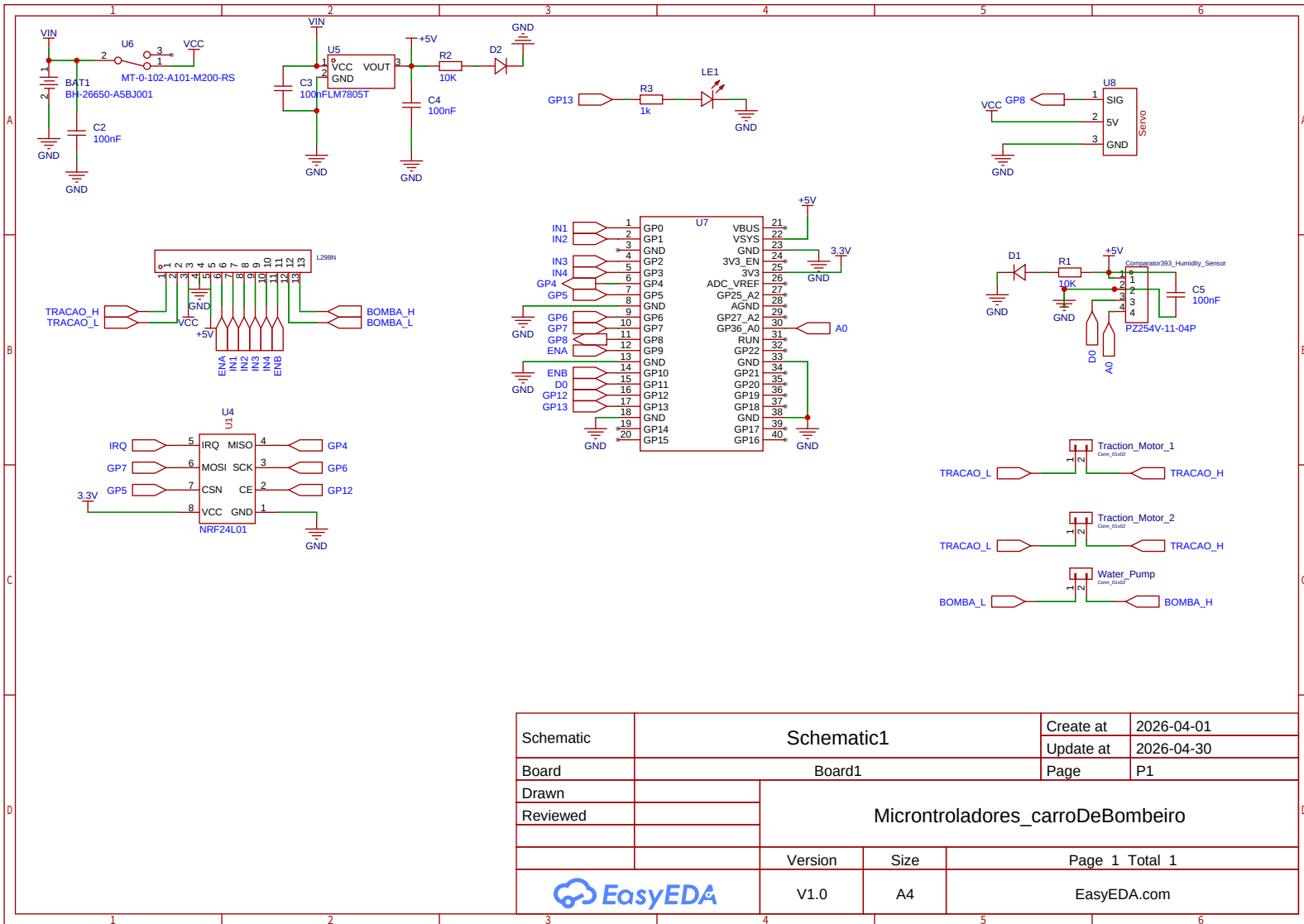
```
273 def normalize(value):
274     return int((value - 32768) / 32768 * 100)
275
276 # =====
277 # SIMULAÇÃO DE UMIDADE
278 # =====
279 # Velocidade de queda: quantos % por segundo segurando o botão.
280 # Ajuste conforme quiser -- ex: 10 = cai 10% a cada segundo.
281 UMIDADE_QUEDA_POR_SEG = 10.0
282
283 umidade_sim = 100          # Começa em 100% e só reseta ao religar
284 btn_anterior = 1          # Estado anterior do botão (1 = solto)
285 btn_pressionado_t = None   # Timestamp de quando começou a pressionar
286
287 def atualizar_umidade_simulada(btn_atual):
288     """
289     Atualiza a umidade simulada com base no estado do botão.
290
291     - btn_atual == 0 → botão pressionado (Pull-up ativo baixo)
292     - btn_atual == 1 → botão solto
293
294     Enquanto pressionado: diminui linearmente pelo tempo.
295     Ao soltar: congela no valor atual.
296     Ao pressionar de novo: continua de onde parou.
297     """
298     global umidade_sim, btn_anterior, btn_pressionado_t
299
300     agora = time.ticks_ms()
301
302     if btn_atual == 0: # Botão pressionado
303         if btn_anterior == 1:
304             # Borda de descida: registra quando começou
305             btn_pressionado_t = agora
306
307             # Calcula quanto tempo está pressionado nesta sessão (ms → s)
308             tempo_pressionado_s = time.ticks_diff(agora, btn_pressionado_t)
309             / 1000.0
310
311             # Calcula nova umidade baseada no tempo acumulado
312             nova = umidade_sim - (UMIDADE_QUEDA_POR_SEG *
313 tempo_pressionado_s)
314             umidade_atual = max(0, int(nova))
315
316         else: # Botão solto
317             if btn_anterior == 0:
318                 # Borda de subida: congela o valor calculado no último frame
319                 agora_antes = btn_pressionado_t
```

```
318         tempo_s = time.ticks_diff(agora, agora_antes) / 1000.0
319         nova = umidade_sim - (UMIDADE_QUEDA_POR_SEG * tempo_s)
320         umidade_sim = max(0, int(nova)) # Salva o valor congelado
321         btn_pressionado_t = None
322
323         umidade_atual = umidade_sim # Retorna o valor congelado
324
325         btn_anterior = btn_atual
326         return umidade_atual
327
328 # =====
329 # LOOP PRINCIPAL
330 # =====
331 INTERVALO_TX = 80
332 JANELA_RX    = 25
333
334 ultimo_tx = time.ticks_ms()
335
336 while True:
337
338     if time.ticks_diff(time.ticks_ms(), ultimo_tx) >= INTERVALO_TX:
339
340         x_norm = -normalize(read_joystick(joy1_x))
341         y_norm = -normalize(read_joystick(joy1_y))
342
343         btn = button.value()
344
345         led.value(1 if btn == 0 else 0)
346
347         # Atualiza umidade simulada
348         umidade_atual = atualizar_umidade_simulada(btn)
349
350         # Determina status (AGUA se >= 30%, SECO abaixo)
351         umidade_status = "AGUA" if umidade_atual >= 30 else "SECO"
352
353         payload = bytearray([
354             x_norm & 0xFF,
355             y_norm & 0xFF,
356             btn,
357             0
358         ])
359
360         radio.send(payload)
361
362         ultimo_tx = time.ticks_ms()
363
364         deadline = time.ticks_add(time.ticks_ms(), JANELA_RX)
```

```
365     recebeu = False
366
367     while time.time_diff(deadline, time.time()) > 0:
368         if radio.available():
369             resp = radio.receive()
370             if resp[2] == 0xFF:
371                 print("RX OK | Umidade real:", resp[0])
372                 recebeu = True
373                 break
374             time.sleep_ms(1)
375
376     if not recebeu:
377         print(f"Sem resposta RF | Umidade SIM: {umidade_atual}% [{umidade_status}]")
378
379     # ---- Atualiza OLED ----
380     display.fill(0)
381
382     display.text("CARRO BOMBEIRO", 4, 0)
383
384     display.text("X:" + str(x_norm), 0, 18)
385     display.text("Y:" + str(y_norm), 0, 30)
386
387     display.text(
388         "BTN:" + ("ON" if btn == 0 else "OFF"),
389         0, 42
390     )
391
392     display.text(
393         "UM:" + str(umidade_atual) + "% " + umidade_status,
394         0, 54
395     )
396
397     display.show()
398
399     time.sleep_ms(1)
```

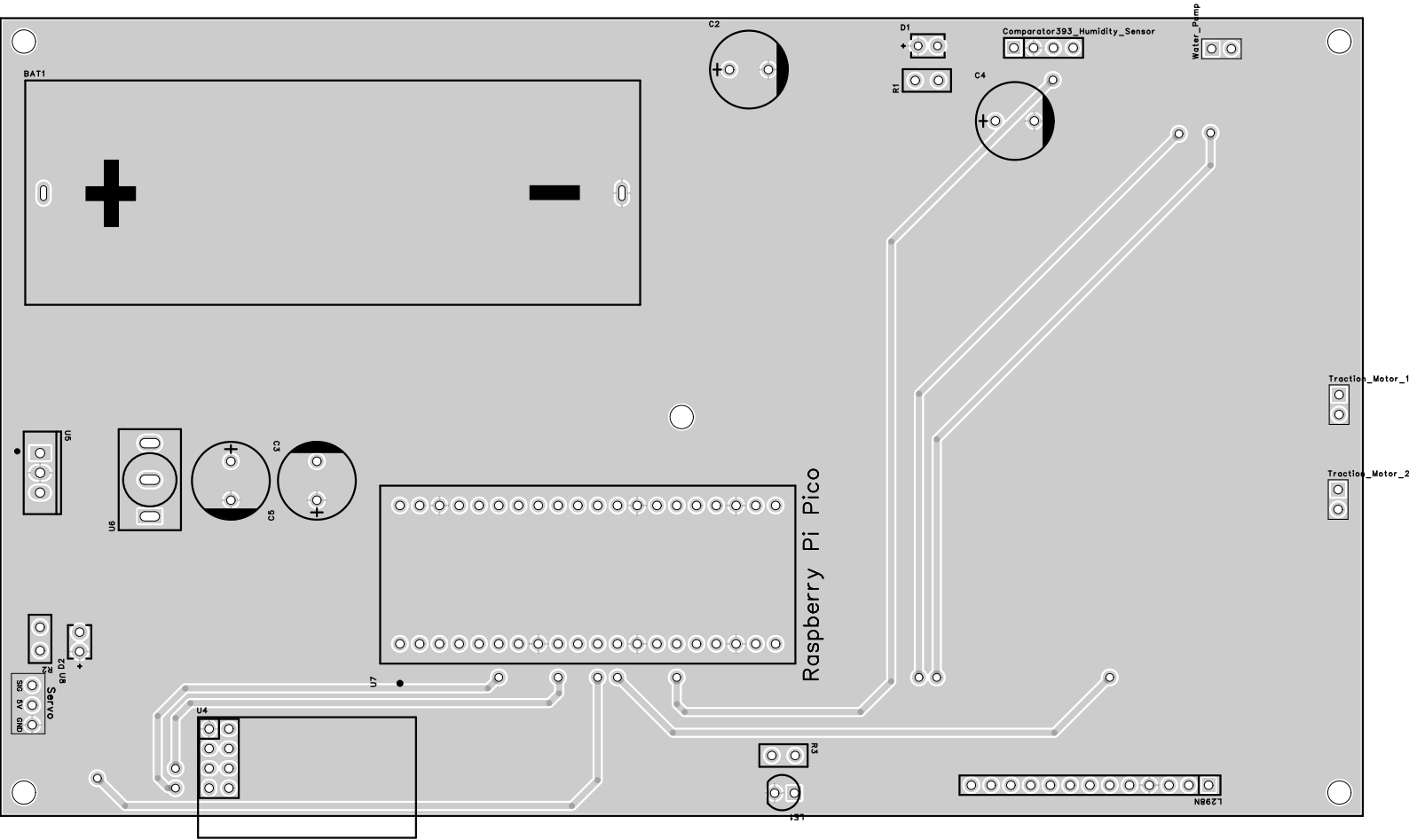
Anexos

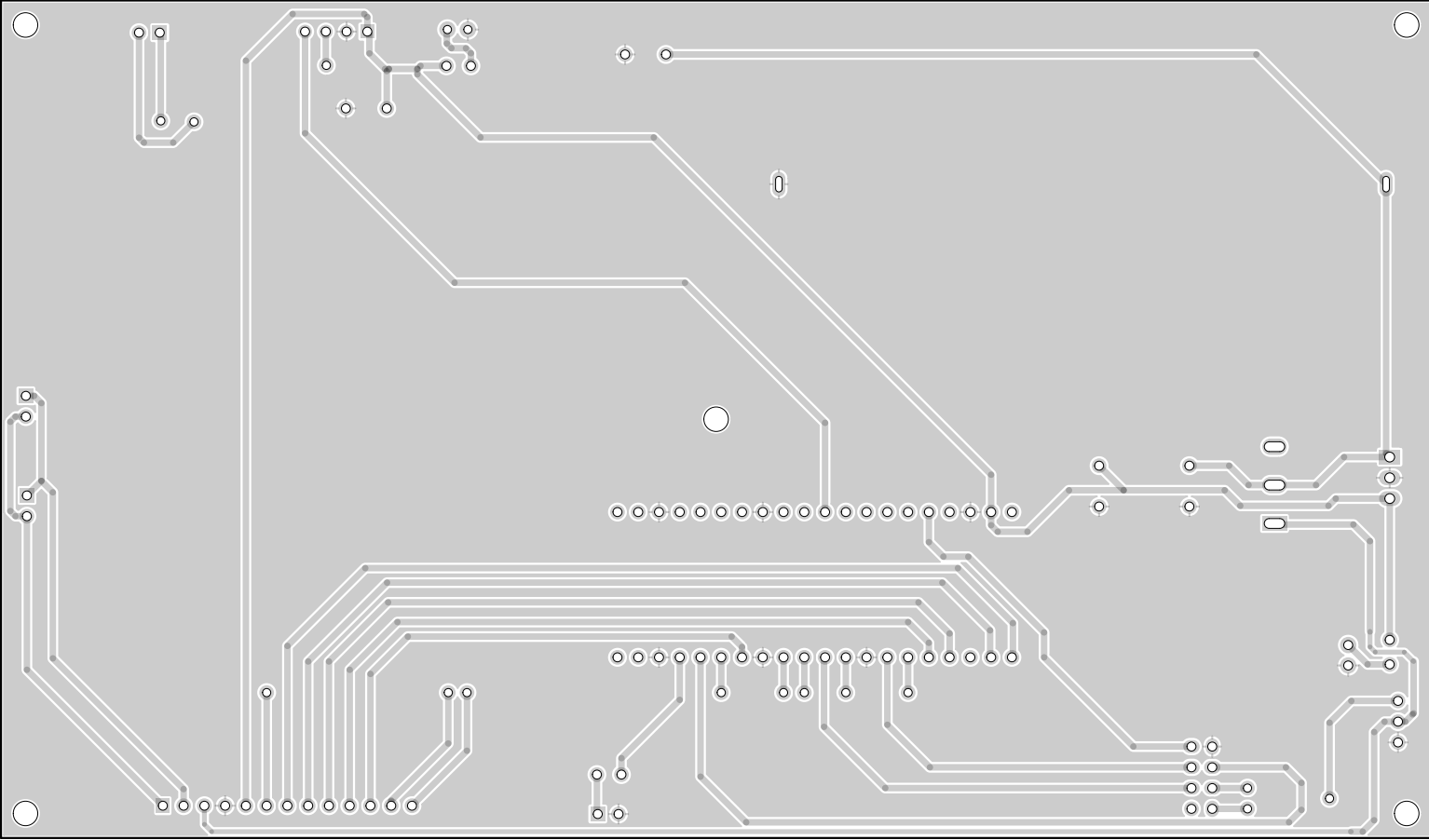
ANEXO A – Esquema Elétrico do Carro



Schematic	Schematic1			Create at	2026-04-01
				Update at	2026-04-30
Board	Board1			Page	P1
Drawn		Microntroladores_carroDeBombeiro			
Reviewed					
		Version	Size	Page 1 Total 1	
		V1.0	A4	EasyEDA.com	

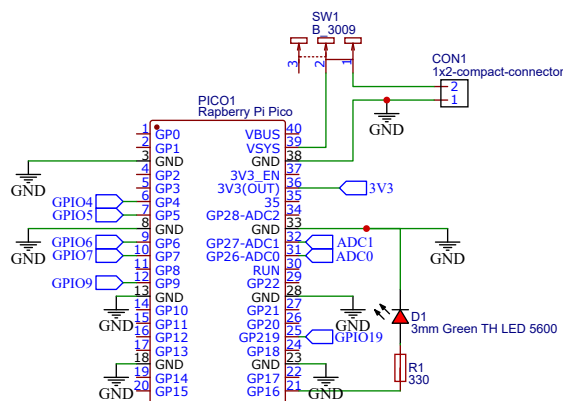
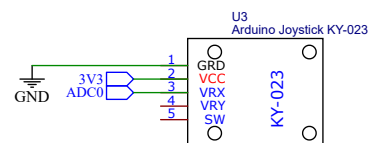
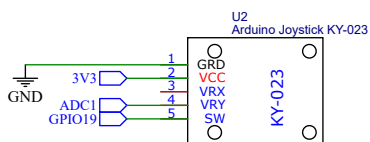
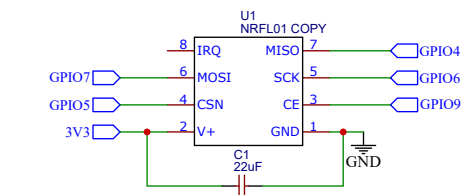
ANEXO B – Layout da PCB do Carro







ANEXO C – Esquema Elétrico do Controle



TITLE: Sheet_1		REV: 1.0
Company: Your Company		Sheet: 1/1
Date: 2026-04-01		Drawn By: jgforuci

ANEXO D – Layout da PCB do Controle

